# STM32基础 - 定时器输入捕获

## 课程目标

1. 熟悉TIM输入捕获原理，实现对硬件全自动测量

## 课程试验

1. 利用STM32完成输入捕获模式测频率试验
2. 利用STM32完成编码器接口测速试验

## 课堂引入

现在随着国家工业4.0的推进，我们看到工厂中出现了越来越多智能机器人替代工人工作，比如汽车生产线上我们可以看到一个个巨型的机械臂灵巧而又高效的完成一个个工件的自动加工任务。那么这些机械臂如何知道自己当前的状态，又如何确保自己每一步的动作都刚好到位呢？这就需要硬件电路不断的对机械臂的位置，角度等基本参数做检测，然后在不断地去修正其运行的误差，才能实现最后的分毫不差的结果。

## 授课进程

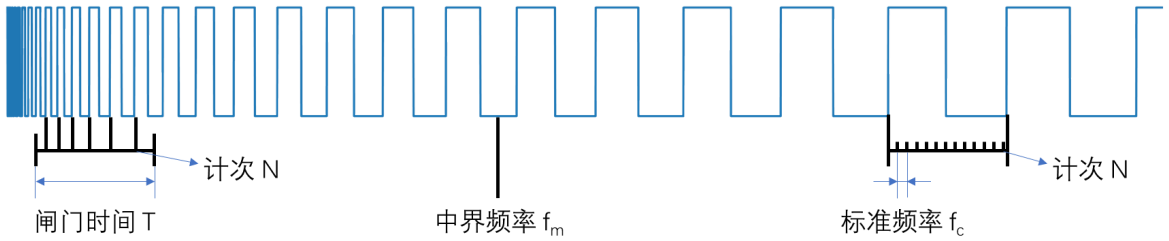
### 一、输入捕获

#### 1、基本介绍

IC（Input Capture）输入捕获。在输入捕获模式下，当通道输入引脚出现指定电平跳变时，当前CNT的值将被锁存到CCR中，可用于测量PWM波形的频率、占空比、脉冲间隔、电平持续时间等参数。每个高级定时器和通用定时器都拥有4个输入捕获通道，而基本定时器没有输入捕获通道。可配置为PWMI模式，同时测量PWM波形的频率和占空比。可配合主从触发模式，实现硬件全自动测量。

#### 2、频率测量

下图信号是一个频率逐渐降低的方波，波形越往左频率越高，越往右频率越低。这个信号都是只有高低电平的数字信号，对于STM32测而言它只能测量数字信号。如果需要测量一个正弦波，那还需要搭建一个信号预处理电路，最简单的方法比如利用运放搭一个比较器，就可以把正弦波转换为数字信号，然后在传给STM32测量。




对于测量频率而言，通常有两种方法可以进行，一是测频法，而是测周法。

测频法就是在闸门时间T内，对上升沿计次，得到N，则频率为 f_x = N / T。

测周法就是在两个上升沿内，以标准频率fc计次，得到N ，则频率为 f_x = f_c / N 。

一般而言，测频法适合测量高频信号，而测周法适合测量低频信号。

那么多高算高，多低算低呢？这就涉及到中界频率的概念了。

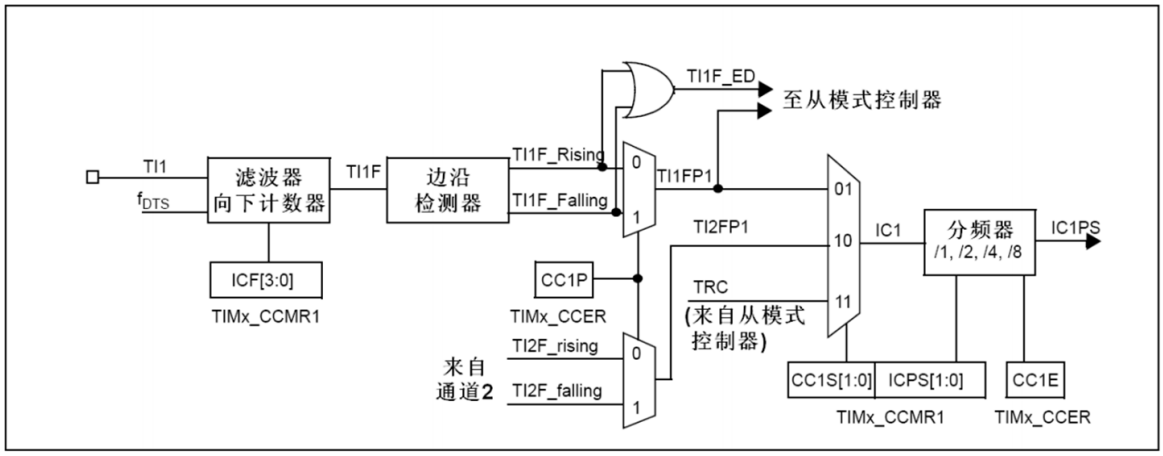
中界频率是指测频法与测周法**误差相等**的频率点。这个频率点可以利用下面的公式来计算：

$$f_m = \sqrt{f_c/T}$$

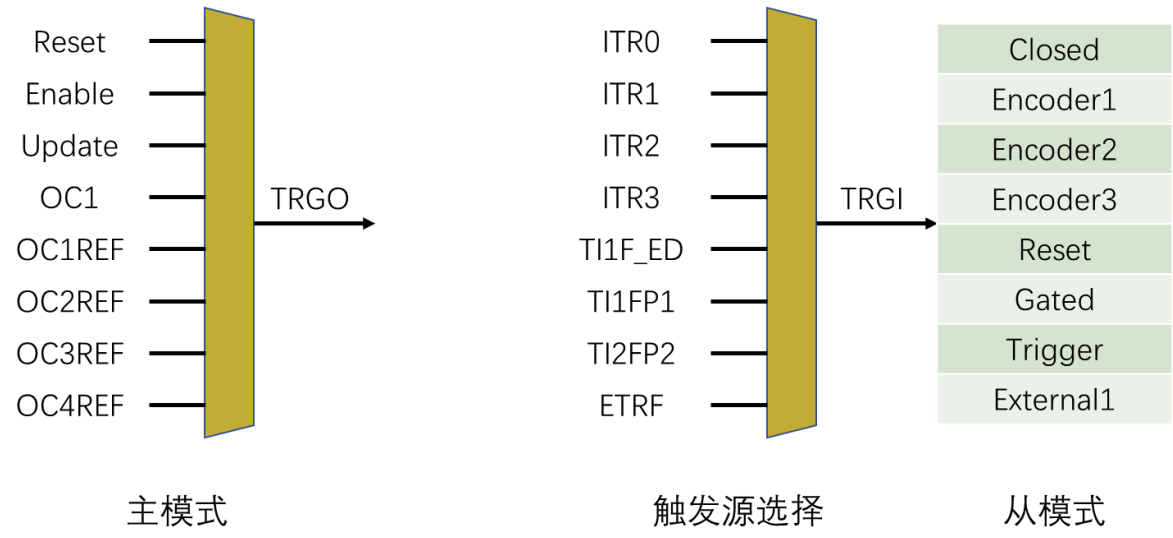
总结一下就是当待测信号频率小于中界频率时，测周法误差更小，选用测周法更合适；当待测信号频率大于中界频率时，测频法误差更小，选用测频法更为合适。

**3、输入捕获通道**

图123　捕获/比较通道(如：通道1输入部分)

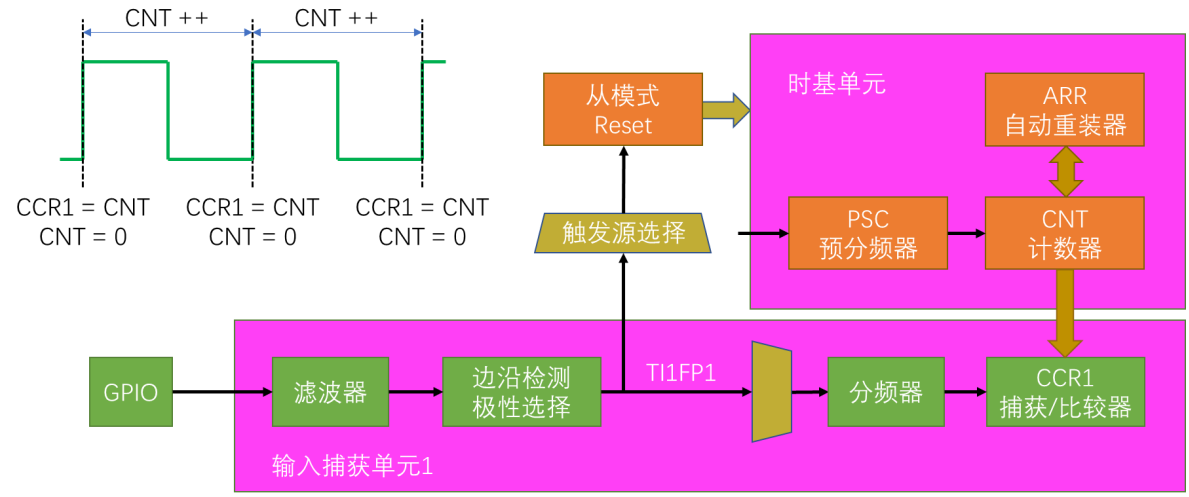



**4、主从触发模式**




主从触发模式就是主模式、从模式和触发源选择这三个功能的简称。其中主模式可以将定时器内部的信号映射到TRGO引脚，用于触发别的外设，所以这部分叫做主模式。从模式就是接收其他外设或者自身外设的一些信号，用于控制自身定时器的运行，也就是被别的信号控制，所以这部分叫从模式。触发源选择，就是选择从模式的触发信号源的，可以认为它是从模式的一部分。触发源选择，选择指定的一个信号，得到TRGI，TRGI去触发从模式。从模式可以在图示列表里选择一项操作来自动执行。

## 5、输入捕获基本结构




这个结构使用了一个通道，目前只能用来测量频率。在右上角是时基单元，配置好时基单元后启动定时器，CNT计数器就会在预分频之后的时钟驱动下不断自增，这个CNT就是利用测周法来计数计时的东西。经过预分频之后，这个位置的时钟频率就是驱动CNT的标准频率fc，这里不难看出标准频率等于72兆除以预分频系数。然后下面输入捕获通道1的GPIO口，输入一个左上角所示的方波信号，经过滤波器和边缘检测，选择TI1FP1为上升沿触发之后，输入选择直连的通道，分频器选择不分频。当TI1FP1出现上升沿之后，CNT的当前计数值转运到CCR1里。同时触发源选择选中TI1FP1为触发信号，从模式选择Reset复位操作，这样TI1FP1的上升沿也会通过上面这一路去触发CNT清0。当然这里会有个先后顺序，一定是先转运CNT的值到CCR里去，然后再触发从模式给CNT清0，或者是非阻塞的同时转移。CNT的值到CCR中，同时0转移到CNT里面去。总之一定不会出现先清0再捕获的情况，要不然捕获值就都是0了。

再来观察一下左上角的图，在信号开始处出现一个上升沿，CCR1等于CNT，就是把CNT的值转运到CCR1里面去，这是输入捕获自动执行的。然后CNT=0，清0计数器，这是从模式自动执行的。在一个周期之内CNT在标准时钟的驱动下不断自增，并且由于之前清0过了，所以CNT就是从上升沿开始从0开始计数，一直++，直到下一次上升沿来临，再执行相同的操作，然后不断重复这个过程。当整个电路工作时，CCR1的值始终保持为最新一个周期的计数值，这个计数值就是n，然后fc/n就是信号的频率，所以当想要读取信号的频率时，只需要读取CCR1得到n，再计算fc/n就可以了。当我们不需要读取的时候，整个电路全自动的测量，不需要占用任何软件资源。

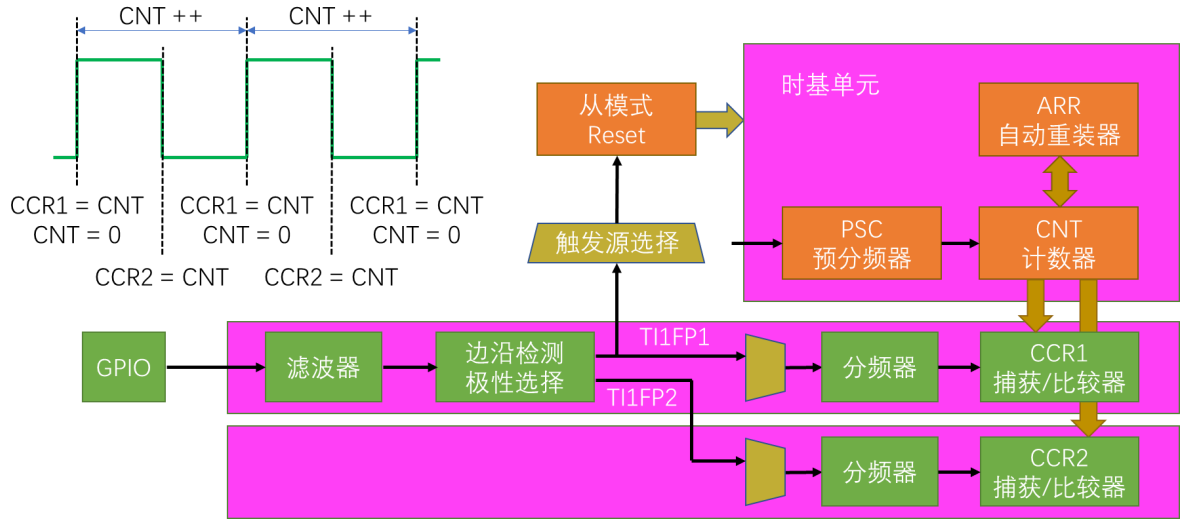
注意事项：

（1）CNT的值是有上限的，ARR一般设置为最大65535，那CNT最大也只能记65535个数，如果信号频率太低，CNT计数值可能会溢

出。

（2）从模式的触发源选择，只有TI1FP1和TI2FP2，没有TI3和TS4的信号，所以如果想使用从模式自动清0CNT，就只能用通道1和通道2。对于通道3和通道4，就只能开启捕获中断，在中断里手动清0了，但是这样程序就会处于频繁中断的状态，比较消耗软件资源。

## 6、PWMI基本结构



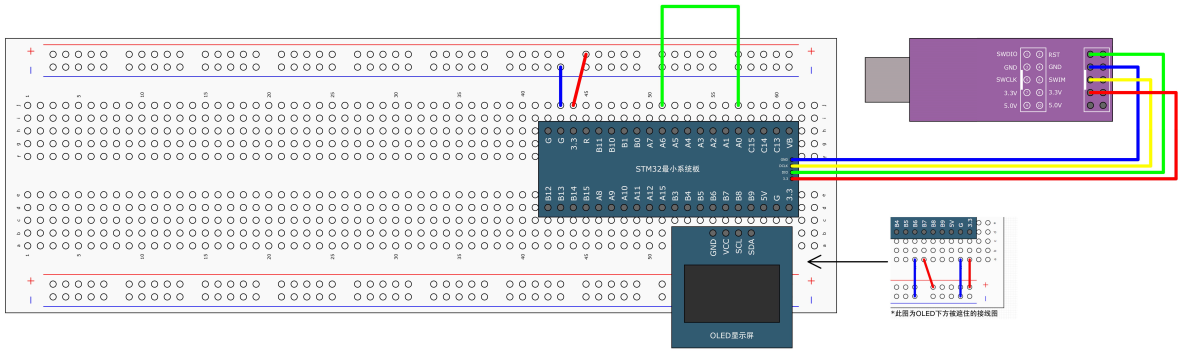

这是PWMI基本结构，使用了两个通道同时捕获一个引脚，可以同时测量周期和占空比。其中上面部分输入捕获基本结构是一样的，而下面部分多了一个通道。首先TI1FP1配置上升沿触发，触发捕获和清零CNT，这是正常的捕获周期。然后再来一个TI1FP2配置为下降沿触发，通过交叉通道去触发通道2的捕获单元。此时会发生什么呢？我们通过左上角的信号图来分析，最开始上升沿CCR1捕获，同时清零CNT，之后CNT一直++，然后在下降沿时刻触发CCR2捕获，所以此时CCR2的值就是CNT在就是高电平期间的计数值。CCR2捕获并不触发CNT清零，所以CNT继续++，直到下一次上升沿，CCR1捕获周期CNT清零。这样执行之后，CCR1就是一整个周期的计数值，CCR2就是高电平期间的计数值。我们用CCR2除以CCR1，就可以获得占空比了。这就是PWMI模式使用两个通道来捕获频率和占空比的思路。这里用两个通道同时捕获第一个引脚的输入，通道2的前面一部分电路就没有用到，当然了，也可以配置两个通道同时捕获第二个引脚的输入，这样就需要使用TI2FP1和TI2FP2这两个引脚了，这两个输入是可以灵活切换的。

## 二、输入捕获实验

（1）实验目的

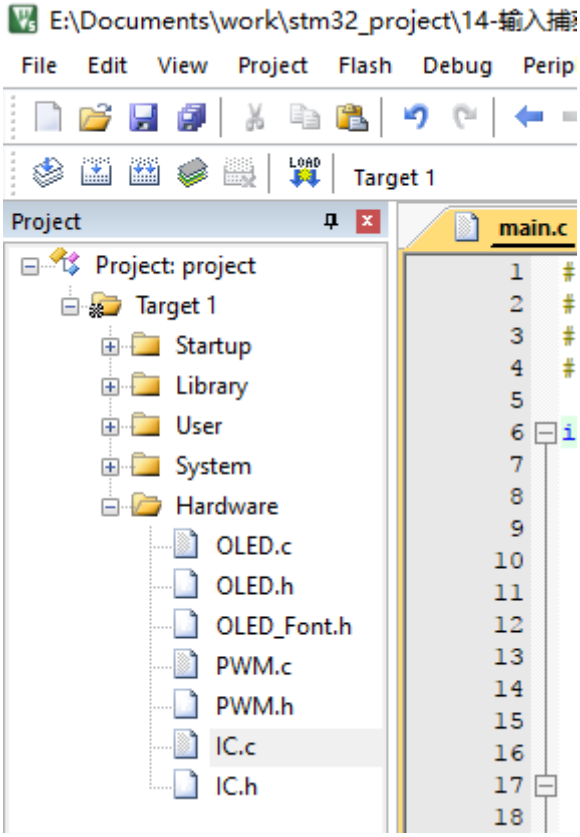
首先通过控制STM32输出PWM波形然后再将这个波形从另外一个引脚输入，利用STM32对这个输入信号频率进行测量最后在OLED显示出测量结果。(用TIM3测试TIM2输出的频率)

（2）硬件电路接线图



（3）程序设计

工程结构如下图：




其中Hardware组与之前工程相比，增加了输入捕获驱动，此外由于PWM波形设置为输出频率为1KHz的信号，所以PWM驱动中的ARR、PSC配置有所变化。具体代码展示如下：

IC.c

```
#include "stm32f10x.h"                  // Device header

void IC_Init(void) {
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE);    // 配置TIM3的时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);   // 配置GPIOA的时钟

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;   // 配置上拉输入
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6;
    GPIO_Init(GPIOA, &GPIO_InitStructure);      // GPIOA初始化

    TIM_InternalClockConfig(TIM3);      //配置使用内部时钟

    TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure;
    TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;     // 配置不分频
    TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;     // 配置向上计数
    TIM_TimeBaseInitStructure.TIM_Period = 65536 - 1;       // ARR
    TIM_TimeBaseInitStructure.TIM_Prescaler = 72 - 1;       // PSC
    TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0;    // 重复计数器，这里没用
    TIM_TimeBaseInit(TIM3, &TIM_TimeBaseInitStructure);     // 初始化TIM3时基单元

    // 输入捕获初始化
    TIM_ICInitTypeDef TIM_ICInitStructure;
    TIM_ICInitStructure.TIM_Channel = TIM_Channel_1;    // 配置定时器通道1
    TIM_ICInitStructure.TIM_ICFilter = 0xF;     // 配置滤波器参数
```

```
    TIM_ICInitStructure.TIM_ICPolarity = TIM_ICPolarity_Rising;     // 配置上升沿触发捕获
    TIM_ICInitStructure.TIM_ICPrescaler = TIM_ICPSC_DIV1;   // 配置不分频
    TIM_ICInitStructure.TIM_ICSelection = TIM_ICSelection_DirectTI;     // 输入信号选择直通
    TIM_ICInit(TIM3, &TIM_ICInitStructure);     // 初始化输入捕获

    TIM_SelectInputTrigger(TIM3, TIM_TS_TI1FP1);    // 触发源选择TI1FP1
    TIM_SelectSlaveMode(TIM3, TIM_SlaveMode_Reset);     // 从模式选择复位，也就是TI1产生上升沿时，会触发CNT清零

    // 使能定时器
    TIM_Cmd(TIM3, ENABLE);
}

uint32_t IC_GetFreq(void) {
    return 1000000 / (TIM_GetCapture1(TIM3) + 1);   // 测周法得到频率fx = fc / N，
                                                    // 这里+1的操作只是为了消除小误差，可以不执行
}
```

IC.h

```
#ifndef __IC_H
#define __IC_H

void IC_Init(void);
uint32_t IC_GetFreq(void);

#endif
```

PWM.c

```
#include "stm32f10x.h"                  // Device header

void PWM_Init(void) {
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    TIM_InternalClockConfig(TIM2);

    TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure;
    TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
    // 这里配置为了输出一个KHz的PWM波形
```

```
    TIM_TimeBaseInitStructure.TIM_Period = 100 - 1;     // ARR
    TIM_TimeBaseInitStructure.TIM_Prescaler = 720 - 1;      // PSC
    TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0;
    TIM_TimeBaseInit(TIM2, &TIM_TimeBaseInitStructure);

    TIM_OCInitTypeDef TIM_OCInitStructure;
    TIM_OCStructInit(&TIM_OCInitStructure);
    TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
    TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;
    TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
    TIM_OCInitStructure.TIM_Pulse = 50; // CCR
    TIM_OC1Init(TIM2, &TIM_OCInitStructure);

    TIM_Cmd(TIM2, ENABLE);
}

void PWM_SetCompare1(uint16_t Compare) {
    TIM_SetCompare1(TIM2, Compare);     // 配置CCR1的值
}

void PWM_SetPerscaler(uint16_t Perscaler) {
    TIM_PrescalerConfig(TIM2, Perscaler, TIM_PSCReloadMode_Immediate);  // 配置PSC的值
}
```

PWM.h

```
#ifndef __PWM_H
#define __PWM_H

void PWM_Init(void);
void PWM_SetCompare1(uint16_t Compare);
void PWM_SetPerscaler(uint16_t Perscaler);

#endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "PWM.h"
#include "IC.h"

int main(void){

    OLED_Init();
    PWM_Init();
    IC_Init();

    OLED_ShowString(1, 1, "Freq:00000Hz");
```

```
    // 使用PWM模块提供输入捕获的测试信号
    PWM_SetPerscaler(720 - 1);  // Freq = 72M / (PSC + 1) / 100
    PWM_SetCompare1(50);        // Duty = CCR / 100

    while(1){
        OLED_ShowNum(1, 6, IC_GetFreq(), 5);     // 不断刷新显示输入捕获测得的频率
    }
}
```

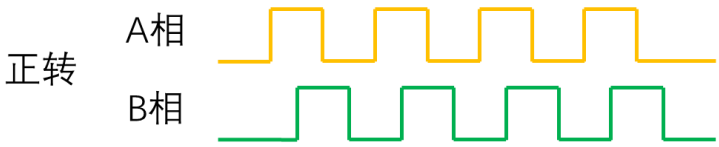
（4）代码完成后，编译并下载到硬件中执行，会发现实验结果正确显示输入波形的频率，符合预期设计。

## 三、编码器接口

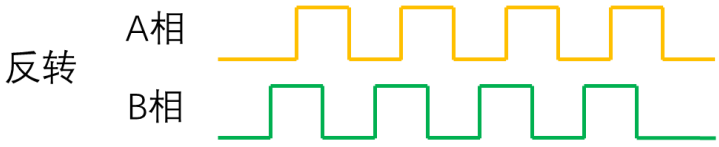
1、基本介绍

编码器接口（Encoder Interface），编码器接口可接收增量（正交）编码器的信号，根据编码器旋转产生的正交信号脉冲，自动控制CNT自增或自减，从而指示编码器的位置、旋转方向和旋转速度。每个高级定时器和通用定时器都拥有1个编码器接口，编码器的资源比较紧张，如果定时器配置了编码器接口，就不能做其他任务了。编码接口的两个输入引脚借用了输入捕获的通道1和通道2。

2、正交编码器




| 边沿 | 另一相状态 |
| --- | --- |
| A相↑ | B相低电平 |
| A相↓ | B相高电平 |
| B相↑ | A相高电平 |
| B相↓ | A相低电平 |



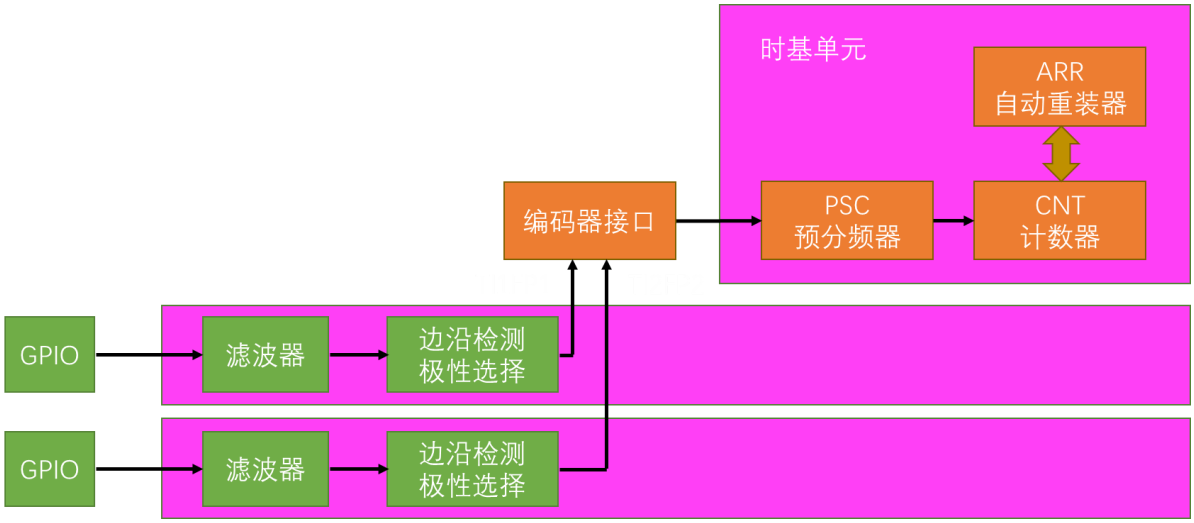

| 边沿 | 另一相状态 |
| --- | --- |
| A相↑ | B相高电平 |
| A相↓ | B相低电平 |
| B相↑ | A相低电平 |
| B相↓ | A相高电平 |

正交编码器一般可以测量位置或者带有方向的速度值，它一般有两个信号输出引脚，一个是A相，一个是B相。

正交信号是如何计次和区分旋转方向呢？首先我们观察一下波形的特点，在正转的时候，第一个时刻A相上升沿，对应B相此时是低电平；继续第2个时刻B相上升沿，对应A相高电平；继续第3个时刻A相下降沿，对应B相高电平；最后是B相下降沿，对应A相低电平；再然后就是A相上升沿，B相低电平了，此时和第一个状态重复了。所以在正转的时候，总结了右边这个表。当出现这些边沿时，对应另一相的状态是表中的四种状态。那么反转的情况呢？第一个时刻B相上升沿，对应A相低电平；第二个时刻A相上升沿，对应B相高电平；第三个时刻B相下降沿，对应A相高电平；第四个时刻A相下降沿，对应B相低电平，这四种状态可以列出下面反转的表。
可以发现，当A、B相出现这些边沿时，对应逆向的状态，正转和反转正好是相反的。比如A相上升沿时，正转B相就是低电平，反转B相就是高电平。其它的也类似，大家可以下来自己再总结。
由于上面的原因，编码器接口的设计逻辑就是首先把A相和B相的所有边沿作为计数器的计数时钟，出现边沿信号时，就计数自增或自减。是增还是减，这个计数的方向由另一相的状态来确定。当出现某个边

沿时，判断另一相的高低电平，如果对应另一相的状态出现在上面正转的表里，那就是正转计时自增；反之，另一相的状态出现在下面反转的表里，那就是反转计时自减。

3、编码器接口基本结构




输入捕获的前两个通道，通过GPIO口接入编码器的A、B相，然后通过滤波器和边沿检测极性选择，产生TI1FP1和TI2FP2，通向编码器接口，编码器接口通过预分频器控制CNT计数器的时钟。同时编码器接口还根据编码器的旋转方向，控制CNT的计数方向。编码器正转时，CNT自增；编码器反转时，CNT自减。此外，ARR也是有效的，一般会设置ARR为65535最大量程。利用补码的特性，很容易得到负数，比如CNT初始为0，正转CNT自增0、1、2、3……等等。反转则CNT自减，就可以得到负数。

4、工作模式

表77　计数方向与编码器信号的关系

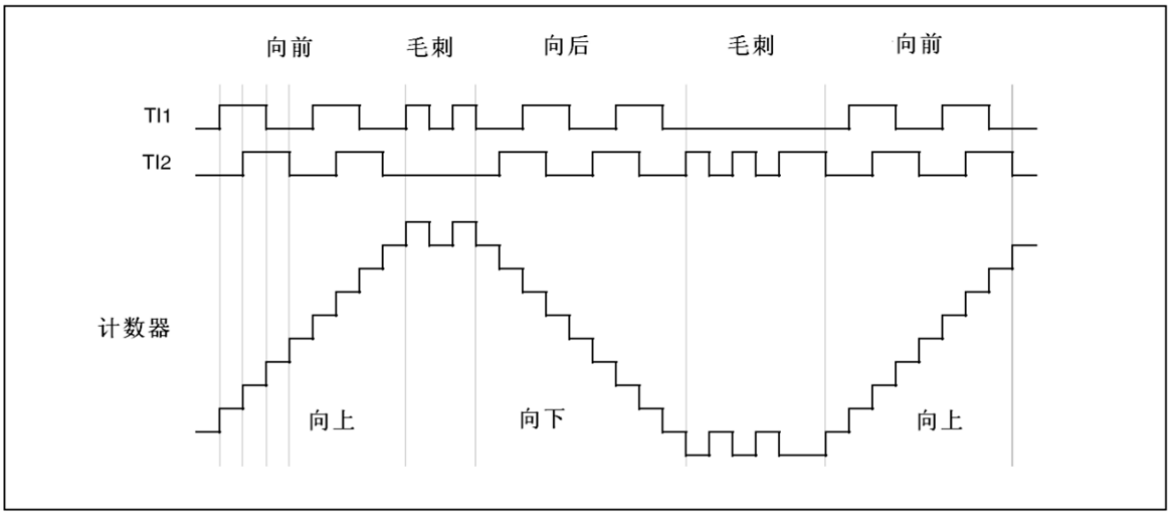
| 有效边沿 | 相对信号的电平 (TI1FP1对应TI2, TI2FP2对应TI1) | TI1FP1信号 | | TI2FP2信号 | |
|---|---|---|---|---|---|
| | | 上升 | 下降 | 上升 | 下降 |
| 仅在TI1计数 | 高 | 向下计数 | 向上计数 | 不计数 | 不计数 |
| | 低 | 向上计数 | 向下计数 | 不计数 | 不计数 |
| 仅在TI2计数 | 高 | 不计数 | 不计数 | 向上计数 | 向下计数 |
| | 低 | 不计数 | 不计数 | 向下计数 | 向上计数 |
| 在TI1和TI2上计数 | 高 | 向下计数 | 向上计数 | 向上计数 | 向下计数 |
| | 低 | 向上计数 | 向下计数 | 向下计数 | 向上计数 |

5、实例

（1）均不反相

| 有效边沿 | 相对信号的电平(TI1FP1对应TI2, TI2FP2对应TI1) | TI1FP1信号 | | TI2FP2信号 | |
|---|---|---|---|---|---|
| | | 上升 | 下降 | 上升 | 下降 |
| 在TI1和TI2上计数 | 高 | 向下计数 | 向上计数 | 向上计数 | 向下计数 |
| | 低 | 向上计数 | 向下计数 | 向下计数 | 向上计数 |

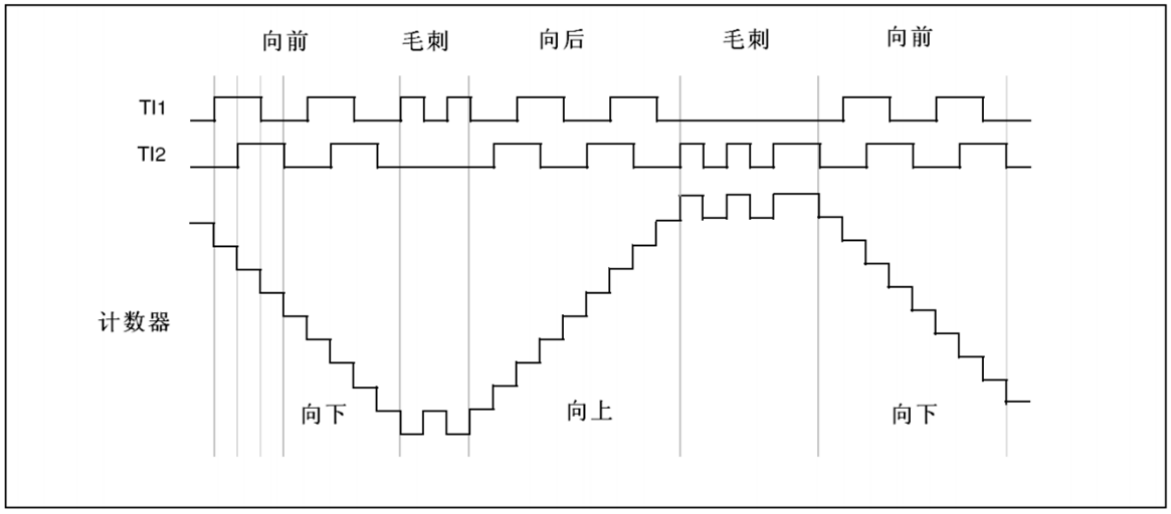
图132　编码器模式下的计数器操作实例




（2）TI1反相

| 有效边沿 | 相对信号的电平(TI1FP1对应TI2, TI2FP2对应TI1) | TI1FP1信号 | | TI2FP2信号 | |
|---|---|---|---|---|---|
| | | 上升 | 下降 | 上升 | 下降 |
| 在TI1和TI2上计数 | 高 | 向下计数 | 向上计数 | 向上计数 | 向下计数 |
| | 低 | 向上计数 | 向下计数 | 向下计数 | 向上计数 |

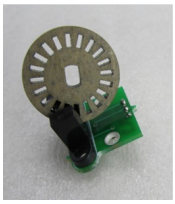
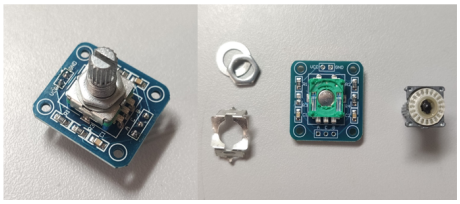
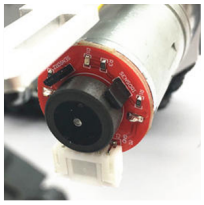
图133　IC1FP1反相的编码器接口模式实例




## 四、旋转编码器

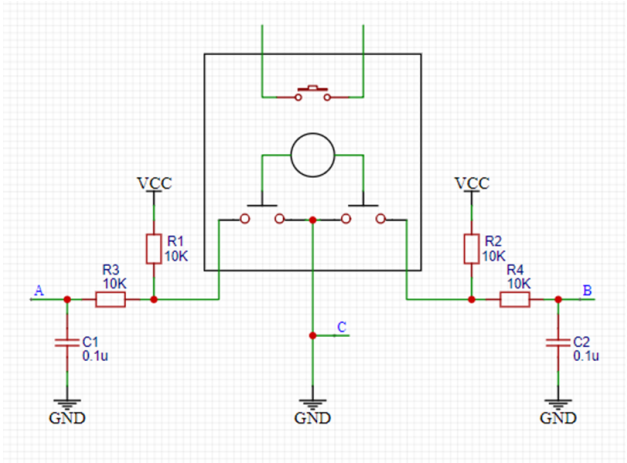
旋转编码器：用来测量位置、速度或旋转方向的装置，当其旋转轴旋转时，其输出端可以输出与旋转速度和方向对应的方波信号，读取方波信号的频率和相位信息即可得知旋转轴的速度和方向。

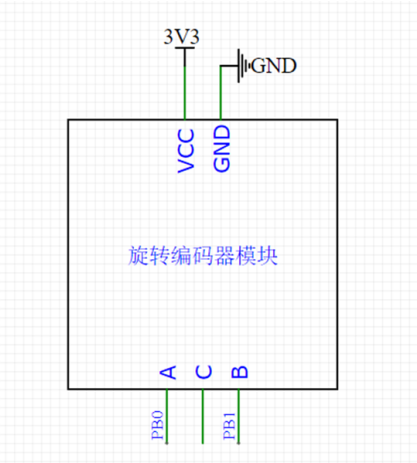
旋转编码器类型包括机械触点式/霍尔传感器式/光栅式等类型。


第一个图是一种最简单的编码器样式，它是使用对测式红外传感器来测速的，为了测速需要配合一个这样的光栅编码盘，当编码盘转动时，红外传感器的红外光就会出现遮挡、透过，遮挡、透过这样的现象对应模块输出的电平，就是高低电平交替的方波，这个方波的个数代表了转过的角度，方波的频率代表转速。然后就可以用编码器接口来捕获这个方波的边缘，以此判断位置和速度。不过这个模块只有一路输出，正转反转输出波形没法区分，所以这种测速方法只能测位置和速度，不能测量旋转方向。
为了进一步测量方向，就可以用后面的几种编码器来实现。第二个图片是旋转编码器，左边是它的外观，右边是它的内部拆解结构。可以看到它内部是用金属触点来进行通断的，因此它是一种机械触点式编码器，左右有两部分开关触点，其中内侧的两根细的触点都是和中间的引脚连接的，而外侧的触点，左边的接到左边的引脚，右边的接到右边的引脚，就是这样的触点连接方式。在正中间有一个圆的金属片是一个按键，旋转编码器的轴是可以按下去的，这个按键的两根线是从上面引出的，按键的轴按下，上面两根线短路，松开上面两根线断开，是一个普通的按键功能。再看一下旋转编码器的编码盘，它也是一系列像光栅一样的东西，只不过它是金属触点，在旋转时依次接通和断开两边的触点。并且还有个关键的部分是这个金属盘的位置是经过设计的，它能让两侧触点的通断产生一个90度的相位差，最终配合外部电路，这个编码器的两个输出就会输出特定的波形。当正转时，左侧的引脚就是A相引脚，输出一个方波信号，同时右侧的引脚就是B相引脚输出一个和它相位相差90度的波形。比如正向旋转时，B相输出是滞后90度的；反向旋转时，A相还是方波信号，而B相就会提前90度，这样正转和反转就可以区分开了。
第三个图是一种直接附在电机后面的编码器，是霍尔传感器形式的编码器。中间是一个圆形磁铁，边上有两个位置错开的霍尔传感器。当磁铁旋转时，通过霍尔传感器就可以输出正交的方波信号了。
第四张图是独立的编码器元件，它的输入轴转动时，输出就会有波形，它也是可以测速和测方向的。具体用法需要再查看相应的手册。
当然第二个图里的编码器，一般是用来进行调节的，比如音响调节音量这样的用途，因为它是触点接触的形式，所以不适合电机这种高速旋转的地方。而另外几种都是非接触的形式，可以用于电机测速，电机测速在电机驱动的应用中还是非常常见的。

## 五、基本电路




左边电路图旋转编码器模块的电路图，中间上部的是旋转编码器上面按键的两根线，在这个模块并没有使用是悬空的。下部是编码器内部的两个触点，旋转轴旋转时，这两个触点以相位相差90度的方式交替导通。因为只是个开关信号，所以要配合外围电路才能输出高低电平。先看图中左边，接了一个10K的上拉电阻，默认没旋转的情况下，这个点被上拉为高电平，通过R~3~电阻输出到A端口就是高电平。当旋转时，内部触点导通，此时该点就直接被拉低到GND，再通过R~3~输出，A端口就是低电平了。图中R~3~是一个输出限流电阻，它是为了防止模块引脚电流过大的，C1是输出滤波电容，可以防止输出信号抖动。右边的电路和左边的是一模一样的，它的输出接到了B端口上。中间这个模块的C端口是直接

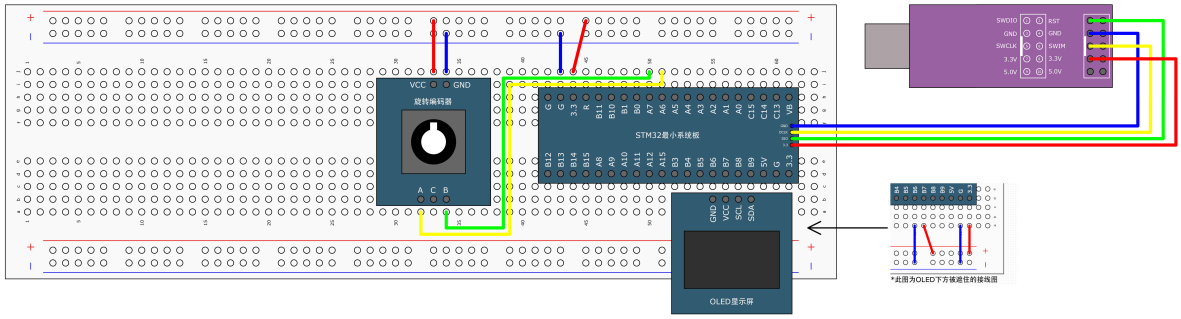
接到了GND的。
右边是使用旋转编码器模块的接线图。上部的VCC和GND接电源，下部的A相输出和B相输出可以直接接到STM32的两个引脚上，比如PB0和PB1。注意引脚的GPIO_Pin编号不要一样就行，然后中间的C引脚也就是GND，暂时不用。

## 六、编码器接口实验

（1）实验目的

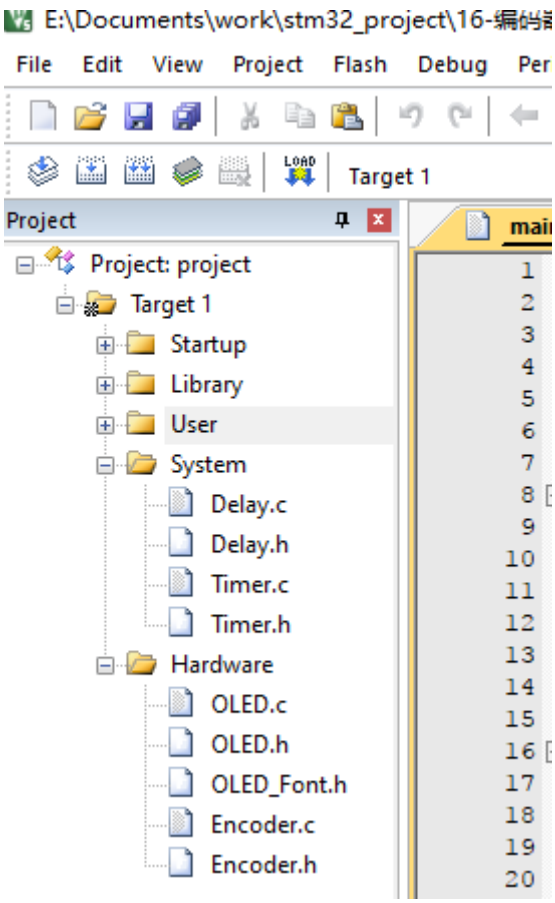
通过控制STM32的定时器编码接口实现旋转编码器自动计次，并将计次结果换算成旋转速度值显示在OLED上。

（2）硬件电路接线图




（3）程序设计

工程结构如下图：




其中Hardware组与之前工程相比，增加了编码器驱动，此外System组增加了定时器驱动，主要配置时基单元。具体代码展示如下：

Encoder.c

```c
#include "stm32f10x.h"                  // Device header

void Encoder_Init(void) {
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE);    // 打开TIM3时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);   // 打开GPIOA时钟

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;   // 配置上拉电阻
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6 | GPIO_Pin_7;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure;
    TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;     // 配置向上计数
    TIM_TimeBaseInitStructure.TIM_Period = 65536 - 1;       // ARR
    TIM_TimeBaseInitStructure.TIM_Prescaler = 1 - 1;        // PSC
    TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0;
    TIM_TimeBaseInit(TIM3, &TIM_TimeBaseInitStructure);     // 初始化TIM3时基单元

    // 输入捕获初始化
    TIM_ICInitTypeDef TIM_ICInitStructure;
    TIM_ICStructInit(&TIM_ICInitStructure);
    TIM_ICInitStructure.TIM_Channel = TIM_Channel_1;    // 配置定时器通道1
    TIM_ICInitStructure.TIM_ICFilter = 0xF;     // 配置滤波器参数
    TIM_ICInit(TIM3, &TIM_ICInitStructure);     // 初始化TIM3的通道1
    TIM_ICInitStructure.TIM_Channel = TIM_Channel_2;    // 配置定时器通道2
    TIM_ICInitStructure.TIM_ICFilter = 0xF;     // 配置滤波器参数
    TIM_ICInit(TIM3, &TIM_ICInitStructure);     // 初始化TIM3的通道2

    // 编码器接口配置
    // Rising配置不反相
    // 注意Encoder配置必须在ICInit方法的下边，否则ICInit的配置就会覆盖Encoder的配置了
    TIM_EncoderInterfaceConfig(TIM3, TIM_EncoderMode_TI12, TIM_ICPolarity_Rising,
TIM_ICPolarity_Rising);

    TIM_Cmd(TIM3, ENABLE);      // 使能定时器
}

int16_t Encoder_Get(void) {
    int16_t temp;
    temp = TIM_GetCounter(TIM3);
    TIM_SetCounter(TIM3, 0);
    return temp;
}
```

Encoder.h

```h
#ifndef __ENCODER_H
#define __ENCODER_H

void Encoder_Init(void);
int16_t Encoder_Get(void);

#endif
```

Timer.c

```c
#include "stm32f10x.h"                  // Device header

void Timer_Init(void) {
    // 设置时钟
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);

    // 设置使用内部定时器
    TIM_InternalClockConfig(TIM2);

    TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure;
    TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;     // 配置向上计数
    // 设定一个1秒的定时，就是定时频率为1Hz，可以参考 定时频率 = 72M/(PSC+1)/(ARR+1) 计算
    // ARR和PSC的取值都应在0~65535范围内
    // Period就是ARR自动重装器的值
    TIM_TimeBaseInitStructure.TIM_Period = 10000 - 1;
    // Prescaler就是PSC预分频器
    TIM_TimeBaseInitStructure.TIM_Prescaler = 7200 - 1;
    // RepetitionCounter是重复计数器，只有高级定时器才需要设置这个
    TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0;
    TIM_TimeBaseInit(TIM2, &TIM_TimeBaseInitStructure);

    // 手动清除更新中断标记位，避免刚上电初始化完就进中断的问题
    TIM_ClearFlag(TIM2, TIM_FLAG_Update);
    // 更新中断到NVIC的通路
    TIM_ITConfig(TIM2, TIM_IT_Update, ENABLE);

    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
    NVIC_InitTypeDef NVIC_InitStructure;
    NVIC_InitStructure.NVIC_IRQChannel = TIM2_IRQn;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 2;
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
    NVIC_Init(&NVIC_InitStructure);     // 初始化NVIC

    // 使能定时器
    TIM_Cmd(TIM2, ENABLE);
}

/*
```

```
void TIM2_IRQHandler(void) {
    if (TIM_GetITStatus(TIM2, TIM_IT_Update) == SET) {

        TIM_ClearITPendingBit(TIM2, TIM_IT_Update);
    }
}
*/
```

Timer.h

```
#ifndef __TIMER_H
#define __TIMER_H

void Timer_Init(void);

#endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "Timer.h"
#include "Encoder.h"

int16_t Speed;

int main(void){

    OLED_Init();
    Timer_Init();
    Encoder_Init();

    OLED_ShowString(1, 1, "Speed:");

    while(1){
        OLED_ShowSignedNum(1, 7, Speed, 5);     // 不断刷新获得编码器的速度值
    }
}

void TIM2_IRQHandler(void) {
    if (TIM_GetITStatus(TIM2, TIM_IT_Update) == SET) {
        Speed = Encoder_Get();      // 每隔固定时间段读取一次编码器计数增量值，也就是速度值
        TIM_ClearITPendingBit(TIM2, TIM_IT_Update);
    }
}
```

（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是旋转编码器快慢，OLED显示的速度值会对应变化，符合预期设计。

## 课程小结

1. STM32的输入捕获利用TIM来实现，配合主从触发模式就可以实现硬件的全自动测量。
2. 编码器的应用非常广泛，比如数据编码，电机控制等都可以利用STM32的编码器接口来实现。正交编码器可以对于位置、速度和旋转方向进行测量。

## 随堂作业

1. 请谈一下输入捕获的频率测量原理。